The code above shows that (on a 64-bit platform) when managed to exceed PHP_FLOAT_MAX, the float $largeFloat has been given a special value of INF (INF is also a built-in constant). Think of this as meaning "too big" rather than true infinity: the number is now simply too big to represent using the IEEE float format. The floating-point specification IEEE-754 uses similar (but more complex) bit trickery to the signed integer: one bit for the sign, 11 bits for the exponent (the powers of ten in standard index), and the remaining bits for the mantissa (the significant digits of the number). Once we've run out of bits, all we can do is flag the value as exceeding limits.

Here are a few examples of how float(INF) behaves in code:

```
$largeFloat = PHP_FLOAT_MAX + 1E292;

var_dump($largeFloat);
var_dump(is_numeric($largeFloat));
var_dump(is_infinite($largeFloat));
var_dump($largeFloat - PHP_FLOAT_MAX);
var_dump($largeFloat / PHP_FLOAT_MAX);
var_dump($largeFloat / $largeFloat);
var_dump($largeFloat / (PHP_FLOAT_MAX + 1E292));

Output:
float(INF)
bool(true)
bool(true)
float(INF)
float(INF)
float(NAN)
float(NAN)
```

We can see that once a number hits float(INF), there's no coming back. Much like how infinity behaves in mathematics. The built-in functions is_numeric() and is_infinite() both return true: the data type is still float, and is_infinite() exists purely to test for floats that have exceeded their bounds.

The last two lines are interesting though: they have the value float(NAN), which stands for "not a number." Like the constant INF, PHP also provides a built-in constant of NAN. Strictly speaking, infinity is not a number either, and either INF or NAN values are an indicator that your code is off the rails. The presence of either in a calculation will result in INF, -INF, NAN, or occasionally zero as a result: